

# **SONiX 32-Bit Cortex-M0 Micro-Controller**

## ***DMA I2C Example Code***

SONiX reserves the right to make change without further notice to any products herein to improve reliability, function or design. SONiX does not assume any liability arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights nor the rights of others. SONiX products are not designed, intended, or authorized for use as components in systems intended, for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SONiX product could create a situation where personal injury or death may occur. Should Buyer purchase or use SONiX products for any such unintended or unauthorized application, Buyer shall indemnify and hold SONiX and its officers, employees, subsidiaries, affiliates and distributors harmless against all claims, cost, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use even if such claim alleges that SONiX was negligent regarding the design or manufacture of the part.



---

---

*AMENDENT HISTORY*

| Version | Date       | Description       |
|---------|------------|-------------------|
| 1.0     | 2024/11/27 | 1. First version. |

---

# Table of Content

---

|                                             |           |
|---------------------------------------------|-----------|
| AMENDENT HISTORY .....                      | 2         |
| <b>1 OVERVIEW.....</b>                      | <b>4</b>  |
| <b>2 FEATURE .....</b>                      | <b>4</b>  |
| <b>3 HW .....</b>                           | <b>4</b>  |
| <b>4 EXAMPLE CODE.....</b>                  | <b>5</b>  |
| 4.1 4 KINDS OF I2C DMA MODE SUPPORTED ..... | 5         |
| 4.2 I2C DMA MASTER TX MODE .....            | 6         |
| 4.3 I2C DMA MASTER RX MODE .....            | 7         |
| 4.4 I2C DMA SLAVE TX MODE .....             | 8         |
| 4.5 I2C DMA SLAVE RX MODE .....             | 9         |
| <b>5 RESULT .....</b>                       | <b>10</b> |

# 1 OVERVIEW

This document provides our customers the C program examples of I2C in DMA mode.

## 2 FEATURE

- I2C DMA Master TX mode example.
- I2C DMA Master RX mode example.
- I2C DMA Slave TX mode example.
- I2C DMA Slave RX mode example.
- Use ARM-compiler V6.12

## 3 HW

- The starter kit board of vendor desired MCU.
- Necessary to pull up I2C SCL and SDA pins with external pull-up resistors.
- Connect I2C pins to another desired I2C master or slave.

# 4 EXAMPLE CODE

## 4.1 4 kinds of I2C DMA mode supported

This example supports 4 basic I2C DMA modes: Master TX, Master Rx, Slave TX and Slave RX. User can select the desired mode by comment out the others.

**I2C Master / Slave TX mode** : This mode will turn on DMA Channel to move data from some peripherals or memory to I2C TX data register. When TX is not empty, DMA request is issued. If the RX terminal responds with NACK, I2CIF will be issued.

**I2C Master / Slave RX mode** : This mode will turn on DMA Channel to move data from I2C RX data register to other peripherals or SRAM. When RX is not empty, DMA request is issued. During DMA transfer, I2C will automatically return ACK to the TX terminal until the transfer is completed.

```
70 //-----
71 //User Code starts HERE!!!
72
73 // Initial I2C
74 I2C0_Init();
75
76 DMAI2C0MasterTX();
77 // DMAI2C0MasterRX();
78 // DMAI2C0SlaveTX();
79 // DMAI2C0SlaveRX();
80
81 while(1); // Pass
82 }
```

## 4.2 I2C DMA Master TX mode

Initial I2C and DMA function. The setting of DMA should refer to the table in the datasheet.

```

70 //-----
71 //User Code starts HERE!!!
72
73 // Initial I2C
74 I2C0_Init();

```

```

92 void DMAI2C0MasterTX(void)
93 {
94     //Initial DMA & I2C for memory to peripheral(I2C TX) transfer
95     DMA_InitSt stDMACH_Init;
96     DMA_StartSt stDMACH_Start;
97     uint32_t eI2C0_DMA_Status = eI2C_DMA_START;
98     uint32_t i;
99
100    // Set DMA CH0 as M(SRAM) to P(I2C TX) mode
101    // Source : Memory, Increment, 8bit, no request, Burst 1
102    stDMACH_Init.b_SrcAddrCtrl = DMA_SRCAD_CTL_INC;
103    stDMACH_Init.b_SrcMode     = DMA_SRC_MEMORY;
104    stDMACH_Init.b_SrcWidth    = DMA_SRC_WIDTH_8BIT;
105    stDMACH_Init.b_SrcReqSel   = DMA_SRCRS_NONE;
106    stDMACH_Init.b_BurstSize   = DMA_BURST_1;
107    // Destination : Peripheral I2C, Fixed, 8bit, I2C0 TX
108    stDMACH_Init.b_DstAddrCtrl = DMA_DSTAD_CTL_FIX;
109    stDMACH_Init.b_DstMode     = DMA_DST_PERIPHERAL;
110    stDMACH_Init.b_DstWidth    = DMA_DST_WIDTH_8BIT;
111    stDMACH_Init.b_DstReqSel   = DMA_DSTRS_I2C;
112    // Channel : LV0, Enable TC/ABT interrupt
113    stDMACH_Init.b_Priority    = DMA_CHPRI_LV0;
114    stDMACH_Init.b_IntTCEn     = DMA_INT_TC_MSK_DIS;
115    stDMACH_Init.b_IntABTEn    = DMA_INT_ABT_MSK_DIS;
116
117    // Initial DMA
118    DMA_Init(&stDMACH_Init, eDMA_CH0);

```

Use I2C to transmit the data from 0x10 to 0x20.

```

125 // Init data buffer
126 for(i = 0; i<DMA_TEST_SIZE ;i++)
127 {
128     // Set Tx data = 0x10, 0x11, ... ,0x20
129     b_aryI2C0Data[i] = (i + 0x10) & 0xFF;
130 }

```

I2C will send the Start & Slave address to the target slave. After receiving the ACK, FW will enable DMA to move data from SRAM to I2C TX data register.

```

135 // DMA Start
136 DMA_Start(&stDMACH_Start, eDMA_CH0);
137
138 I2C0_DMA_Master_TX_Start(I2C_ADDR_SLAVE_ADDR0, DMA_TEST_SIZE);

```

### 4.3 I2C DMA Master RX mode

Initial I2C and DMA function. The setting of DMA should refer to the table in the specifications.

```

70 //-----
71 //User Code starts HERE!!!
72
73 // Initial I2C
74 I2C0_Init();

```

```

167 void DMAI2C0MasterRX(void)
168 {
169     //Initial DMA & I2C for peripheral(I2C RX) to memory transfer
170     DMA_InitSt stdMACH_Init;
171     DMA_StartSt stdMACH_Start;
172     uint32_t eI2C0_DMA_Status = eI2C_DMA_START;
173     uint32_t i;
174
175     // Set DMA CH0 as P(I2C RX) to M(SRAM) mode
176     // Source : Peripheral I2C, Fixed, 8bit, I2C0 TX
177     stdMACH_Init.b_SrcAddrCtrl = DMA_SRCAD_CTL_FIX;
178     stdMACH_Init.b_SrcMode     = DMA_SRC_PERIPHERAL;
179     stdMACH_Init.b_SrcWidth    = DMA_SRC_WIDTH_8BIT;
180     stdMACH_Init.b_SrcReqSel   = DMA_SRCRS_I2C;
181     stdMACH_Init.b_BurstSize   = DMA_BURST_1;
182     // Destination : Memory, Increment, 8bit, no request, Burst 1
183     stdMACH_Init.b_DstAddrCtrl = DMA_DSTAD_CTL_INC;
184     stdMACH_Init.b_DstMode     = DMA_DST_MEMORY;
185     stdMACH_Init.b_DstWidth    = DMA_DST_WIDTH_8BIT;
186     stdMACH_Init.b_DstReqSel   = DMA_DSTRS_NONE;
187     // Channel : LV0, Enable TC/ABT interrupt
188     stdMACH_Init.b_Priority    = DMA_CHPRI_LV0;
189     stdMACH_Init.b_IntTCEn     = DMA_INT_TC_MSK_DIS;
190     stdMACH_Init.b_IntABTEn    = DMA_INT_ABT_MSK_DIS;
191
192     // Initial DMA
193     DMA_Init(&stdMACH_Init, eDMA_CH0);

```

I2C will send the Start & Slave address to the target slave. After receiving the ACK, FW will enable DMA to move data from I2C RX data register to SRAM.

```

209 // DMA Start
210 DMA_Start(&stdMACH_Start, eDMA_CH0);
211
212 I2C0_DMA_Master_RX_Start(I2C_ADDR_SLAVE_ADDR0, DMA_TEST_SIZE, I2C_RX_DMA_LAST_NACK);

```

## 4.4 I2C DMA Slave TX mode

Initial I2C and DMA function. The setting of DMA should refer to the table in the specifications.

```

70 //-----
71 //User Code starts HERE!!!
72
73 // Initial I2C
74 I2C0_Init();

```

```

247 void DMAI2C0SlaveTX(void)
248 {
249     //Initial DMA & I2C for memory to peripheral(I2C TX) transfer
250     DMA_InitSt stDMACH_Init;
251     DMA_StartSt stDMACH_Start;
252     uint32_t eI2C0_DMA_Status = eI2C_DMA_START;
253     uint32_t i;
254
255     // Set I2C slave address
256     I2C0_SlaveAddressSet(I2C_ADD_MODE_7BIT, I2C_GCEN_DIS);
257
258     // Set DMA CH0 as M(SRAM) to P(I2C TX) mode
259     // Source : Memory, Increment, 8bit, no request, Burst 1
260     stDMACH_Init.b_SrcAddrCtrl = DMA_SRCAD_CTL_INC;
261     stDMACH_Init.b_SrcMode     = DMA_SRC_MEMORY;
262     stDMACH_Init.b_SrcWidth    = DMA_SRC_WIDTH_8BIT;
263     stDMACH_Init.b_SrcReqSel   = DMA_SRCRS_NONE;
264     stDMACH_Init.b_BurstSize   = DMA_BURST_1;
265     // Destination : Peripheral I2C, Fixed, 8bit, I2C0 TX
266     stDMACH_Init.b_DstAddrCtrl = DMA_DSTAD_CTL_FIX;
267     stDMACH_Init.b_DstMode     = DMA_DST_PERIPHERAL;
268     stDMACH_Init.b_DstWidth    = DMA_DST_WIDTH_8BIT;
269     stDMACH_Init.b_DstReqSel   = DMA_DSTRS_I2C;
270     // Channel : LV0, Enable TC/ABT interrupt
271     stDMACH_Init.b_Priority    = DMA_CHPRI_LV0;
272     stDMACH_Init.b_IntTCEn     = DMA_INT_TC_MSK_DIS;
273     stDMACH_Init.b_IntABTEn    = DMA_INT_ABT_MSK_DIS;

```

Use I2C to transmit the data from 0x10 to 0x20.

```

283 // Init data buffer
284 for(i = 0; i<DMA_TEST_SIZE ;i++)
285 {
286     // Set Tx data = 0x10, 0x11, ... ,0x20
287     b_aryI2C0Data[i] = (i + 0x10) & 0xFF;
288 }

```

As an I2C slave, the MCU should wait for the master to send start and slave address . After receiving the Slave TX Hit, FW will enable DMA to move data from SRAM to I2C TX data register.

```

293 // DMA Start
294 DMA_Start(&stDMACH_Start, eDMA_CH0);
295
296 I2C0_DMA_Slave_TX_Start(DMA_TEST_SIZE);

```



## 4.5 I2C DMA Slave RX mode

Initial I2C and DMA function. The setting of DMA should refer to the table in the specifications.

```

70 //-----
71 //User Code starts HERE!!!
72
73 // Initial I2C
74 I2C0_Init();

```

```

325 void DMAI2C0SlaveRX(void)
326 {
327     //Initial DMA & I2C for peripheral(I2C RX) to memory transfer
328     DMA_InitSt stDMACH_Init;
329     DMA_StartSt stDMACH_Start;
330     uint32_t eI2C0_DMA_Status = eI2C_DMA_START;
331     uint32_t i;
332
333     // Set I2C slave address
334     I2C0_SlaveAddressSet(I2C_ADD_MODE_7BIT, I2C_GCEN_DIS);
335
336     // Set DMA CH0 as P(I2C RX) to M(SRAM) mode
337     // Source : Peripheral I2C, Fixed, 8bit, I2C0 TX
338     stDMACH_Init.b_SrcAddrCtrl = DMA_SRCAD_CTL_FIX;
339     stDMACH_Init.b_SrcMode     = DMA_SRC_PERIPHERAL;
340     stDMACH_Init.b_SrcWidth    = DMA_SRC_WIDTH_8BIT;
341     stDMACH_Init.b_SrcReqSel   = DMA_SRCRS_I2C;
342     stDMACH_Init.b_BurstSize   = DMA_BURST_1;
343     // Destination : Memory, Increment, 8bit, no request, Burst 1
344     stDMACH_Init.b_DstAddrCtrl = DMA_DSTAD_CTL_INC;
345     stDMACH_Init.b_DstMode     = DMA_DST_MEMORY;
346     stDMACH_Init.b_DstWidth    = DMA_DST_WIDTH_8BIT;
347     stDMACH_Init.b_DstReqSel   = DMA_DSTRS_NONE;
348     // Channel : LV0, Enable TC/ABT interrupt
349     stDMACH_Init.b_Priority    = DMA_CHPRI_LV0;
350     stDMACH_Init.b_IntTCEn     = DMA_INT_TC_MSK_DIS;
351     stDMACH_Init.b_IntABTEn    = DMA_INT_ABT_MSK_DIS;
352
353     // Initial DMA
354     DMA_Init(&stDMACH_Init, eDMA_CH0);

```

As an I2C slave, the MCU should wait for the master to send start and slave address . After receiving the Slave RX Hit, FW will enable DMA to move data from I2C RX data register to SRAM.

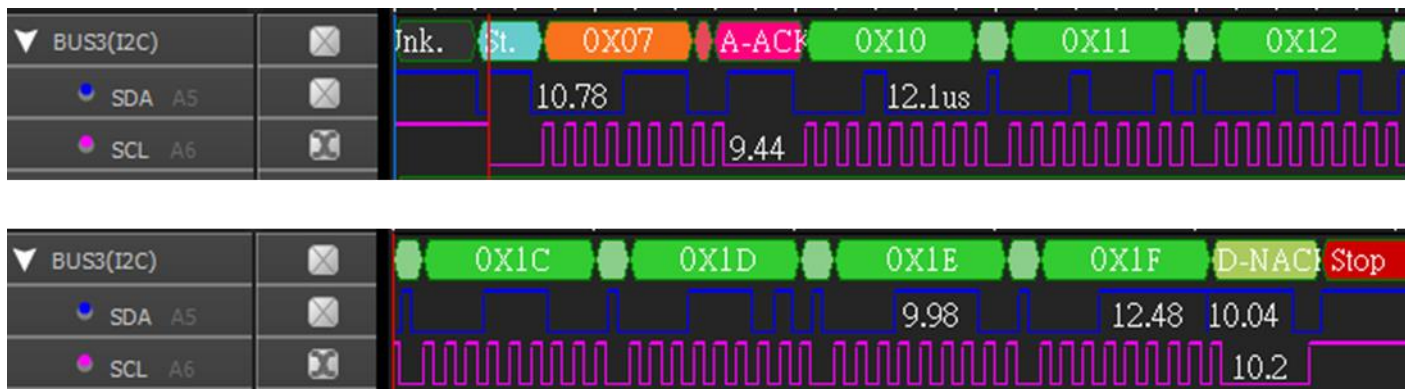
```

370 // DMA Start
371 DMA_Start(&stDMACH_Start, eDMA_CH0);
372
373 I2C0_DMA_Slave_RX_Start(DMA_TEST_SIZE, I2C_RX_DMA_LAST_NACK);

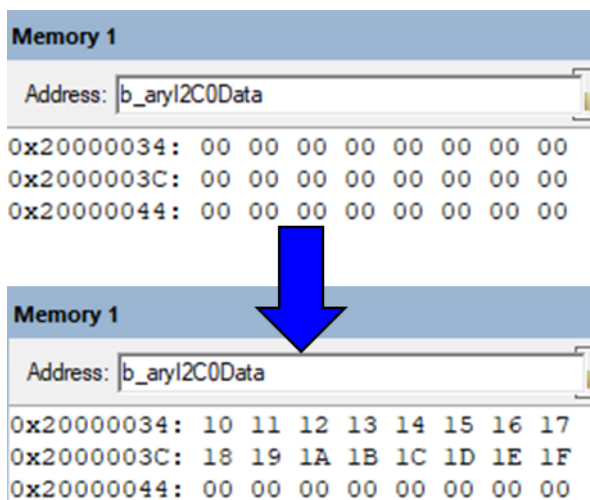
```

# 5 RESULT

The data sent or received can be observed on I2C bus.



If it is master RX mode or slave RX mode, the results can be observed through Watch window or Memory window.



SONiX reserves the right to make change without further notice to any products herein to improve reliability, function or design. SONiX does not assume any liability arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights nor the rights of others. SONiX products are not designed, intended, or authorized for use as components in systems intended, for surgical implant into the body, or other applications intended to support or sustain life, or for any other application in which the failure of the SONiX product could create a situation where personal injury or death may occur. Should Buyer purchase or use SONiX products for any such unintended or unauthorized application. Buyer shall indemnify and hold SONiX and its officers, employees, subsidiaries, affiliates and distributors harmless against all claims, cost, damages, and expenses, and reasonable attorney fees arising out of, directly or indirectly, any claim of personal injury or death associated with such unintended or unauthorized use even if such claim alleges that SONiX was negligent regarding the design or manufacture of the part.

**Main Office:**

Address: 10F-1, NO. 36, Taiyuan Street, Chupei City, Hsinchu, Taiwan R.O.C.  
Tel: 886-3-5600 888  
Fax: 886-3-5600 889

**Taipei Office:**

Address: 15F-2, NO. 171, Song Ted Road, Taipei, Taiwan R.O.C.  
Tel: 886-2-2759 1980  
Fax: 886-2-2759 8180

**Hong Kong Office:**

Unit No.705, Level 7 Tower 1, Grand Central Plaza 138 Shatin Rural Committee Road, Shatin, New Territories, Hong Kong.  
Tel: 852-2723-8086  
Fax: 852-2723-9179

**Technical Support by Email:**

Sn8fae@sonix.com.tw